环境启用与数据处理指南

版本	修改点	作者	工号
V1.0	初版文档撰写	陈怡恒	cWX1501401

2026 年 8 月 26 日

目　录

第 1 章 编写目的与使用范围 **1**

第 2 章 环境部署 **2**

2.1 FSDB 格式转换环境部署 2

2.1.1 动态链接库与编译环境 2

2.1.2 Python 环境 2

2.2 代理筛选环境部署 3

第 3 章 代理筛选流程快速启动 **4**

3.1 数据集提取快速启动 4

3.2 代理筛选快速启动 5

附录 A 数据集提取脚本 **8**

A.1 fsdb_to_tog_cache 8

A.2 fsdb_to_power_label 与 TSV 转 NPY 过程 10

A.3 merge_tog_cache_manifests.py 12

A.4 dimentionality_reduction_csv.py 13

附录 B 代理筛选脚本 **16**

B.1 step1_make_global_sample_index.py 16

B.2 step1_prepare_timewindow.py 17

B.3 step2_mcp_per_offset.py 19

B.4 step2_xgb_per_offset.py 20

B.5 xgb_power.py 22

附录 C 相关文档 **25**

第 1 章　编写目的与使用范围

本文档用于说明数据处理与模型训练过程所需依赖环境的搭建方法，并给出从原始数据到训练数据集的提取边界。完成部署后，使用者应能够运行快速信号数据库（FSDB）格式转换程序，生成模型所需的特征与标签，并完成代理特征筛选环境的部署。

为兼顾快速使用与细节查阅，正文部分给出必要的环境初始化方法，以及数据集提取和代理筛选快速启动脚本的总体流程与启动方式；具体的脚本功能、参数定义、输入输出件、使用约束和验收要求等细节在附录中展开。代理筛选算法的原理与完整执行流程另见《代理筛选算法说明》。

Rocket–Gemmini 硬件平台的生成、仿真与部署可参考 Chipyard 官方文档，本文不重复介绍相关硬件工具链。项目后端环境基于华为 Dophis 平台搭建，相关技术资料已较为完善，本文不再展开说明。

本文档仅适用于功耗预测任务与电压降风险预测任务中，数据处理与模型训练环境准备阶段，内容包括：

- 快速信号数据库（FSDB）格式转换所需的动态链接库、编译工具和 Python 环境；
- 时间窗口样本构造、MCP/XGBoost 代理筛选所需的 Python 环境；
- 电平/翻转特征矩阵、功耗标签及电压降风险标签的数据提取与构造流程。

本文不涉及 Rocket–Gemmini 硬件平台部署、Chipyard 工具链安装及华为 Dophis 平台初始化，也不展开模型训练、超参数优化和后端物理设计等流程。受数据集构建进度限制，电压降风险标签转换目前仍处于接口与算法设计阶段，尚未形成经过完整验证的处理流程。

第 2 章　环境部署

本章说明首次运行前必须具备的两套环境。路径示例均需替换为目标机器上的实际绝对路径；批处理脚本建议显式传入 PYTHON，避免登录环境与计算节点解释器不一致。

2.1　FSDB 格式转换环境部署

FSDB 格式转换同时依赖 FsdbReader 动态链接库、C++ 编译环境和 Python 数据处理环境。

2.1.1　动态链接库与编译环境

目标机器应提供支持 C++17 的 g++、make，以及可用的 Verdi 2018.09sp2/FsdbReader 安装和配套许可证环境。设置软件包根目录并编译：

```
export PACKAGE_ROOT=/path/to/xgboost_timewindow
cd "${PACKAGE_ROOT}/src/fsdbapi"

# 使用工程内置 FsdbReader 配置
make

# 或显式指定站点 Verdi
make VERDI_HOME=/path/to/verdi/2018.09sp2
```

编译后确认两个转换程序存在，且不存在未解析的动态库：

```
test -x build/fsdb_to_tog_cache
test -x build/fsdb_to_power_label
ldd build/fsdb_to_tog_cache | grep 'not found'
```

最后一条命令应无输出。还应选取一个实际 FSDB 文件执行最小读取测试，以同时验证动态库版本、FSDB 文件格式和许可证环境。

2.1.2　Python 环境

格式转换后的缓存合并、数组转换和特征清洗至少需要 numpy、scipy 与 pandas。项目交付文件中附带已经验证的 Conda 环境压缩包 pro.tar.gz；环境初始化时，将该文件解压到指定目录并修正环境前缀即可使用，无需重新安装 Python 及相关依赖。

```
# 将交付文件 pro.tar.gz 解压到指定位置
mkdir -p /path/to/conda/envs/pro
tar -xzf /path/to/delivery/pro.tar.gz -C /path/to/conda/envs/pro

# 激活环境并修正打包前缀
source /path/to/conda/envs/pro/bin/activate
```

```
conda-unpack

export CONDA_ENV_ROOT=/path/to/conda/envs/pro
export PYTHON=${CONDA_ENV_ROOT}/bin/python
"${PYTHON}" --version
```

后续批处理任务可不激活交互式 Conda 环境，但必须显式设置 PYTHON，确保计算节点使用上述交付环境中的解释器。

2.2 代理筛选环境部署

交付文件中的 pro.tar.gz 已包含代理筛选与验证所需的 numpy、scipy、pandas、scikit-learn、xgboost、optuna、matplotlib、joblib 和 skglm 等依赖，无需另行安装。按照 2.1.2 节完成解压和前缀修正后，激活 pro 环境并执行以下命令检查环境完整性：

```
source /path/to/conda/envs/pro/bin/activate
python -m pip check

python - <<'PY'
import sys
import numpy, scipy, pandas, sklearn, xgboost, optuna
import matplotlib, joblib, skglm
print("python", sys.executable)
print("xgboost", xgboost.__version__)
print("skglm", getattr(skglm, "__version__", "unknown"))
print("proxy selection environment: OK")
PY
```

python-mpipcheck 应输出 Nobrokenrequirementsfound.，Python 导入检查应正常结束并打印 proxyselectionenvironment:OK。若检查失败，应首先确认当前解释器来自已解压的 pro 环境，并检查是否已执行 conda-unpack。

第 3 章　代理筛选流程快速启动

3.1　数据集提取快速启动

cmd/run_fsdb_pipeline.csh 是当前数据集提取阶段的总控入口。该脚本已实现并验证功耗预测任务的数据处理流程：从 RTL FSDB 导出信号电平/翻转信息，从 PTPX 功耗 FSDB 导出功耗标签，随后完成跨用例合并和无效信息剔除。电压降风险预测任务可复用其中的特征导出、缓存合并和无效信息剔除流程，但受项目阶段与数据采集时间限制，电压降报告及标签尚未完整收集，当前尚未形成经过完整验证的端到端流程。因此，本节将现有脚本作为电压降风险预测数据流的可迁移基线，而不将其表述为已经闭环的电压降数据流。

数据集处理采用分阶段组织方式，各阶段通过文件和 manifest 传递结果，特征提取与标签生成相互解耦。迁移到电压降风险预测标签任务时，无需重写 FSDB 特征处理链，只需替换任务相关的标签生成阶段，并保持用例标识、采样行数和时间对齐关系一致。

启动命令示例：

```
cd /path/to/xgboost_timewindow
csh cmd/run_fsdb_pipeline.csh gemmini
```

示例中的 gemmini 为硬件或数据集标识 HW。脚本依次完成信号过滤、FSDB 特征分块导出、功耗标签导出、跨用例缓存合并和无效信息剔除。首次运行前应重点检查下表中的配置。

类别	变量或配置	检查内容
运行环境	activate.csh、LD_LIBRARY_PATH	确认 Verdi 2018.09sp2/FsdbReader 动态库和许可证可用，并已激活交付的 pro 环境。
输入与输出	FSDB_DIR、OUTPUT_BASE、WORK_DIR	确认输入路径、输出容量和计算节点访问权限；同名 HW 会复用固定输出位置。
RTL 特征输入	fsdb_list	依次填写用例标识、RTL FSDB、采样起点和终点。用例标识须唯一，且终点大于起点。
功耗标签输入	power_list	依次填写与 fsdb_list 对应的用例标识、功耗 FSDB 和信号层级；不导出功耗时设为空列表。

采样设置	STEP、END_INCL、DROP_FIRST	起点、终点和间隔均使用 FSDB tag，实际物理时间由波形文件的仿真 timescale 决定。
信号与分块	SIGNAL_PREFIX、RAW_COL、MAX_CYCLES_PER_FILE	检查层级过滤范围、每个原始信号块的规模和 NPZ 分片行数。
资源与调度	MAX_MEM、DSUB_QUEUE、DSUB_EXTRA_ARGS、POLL_SEC	根据后端调度平台调整队列、内存申请和轮询周期。

注意：脚本会删除并重建 WORK_DIR/cache_HW，并清除同名流程的部分 complete/failed 标志。重新运行前须确认 HW、WORK_DIR 和 OUTPUT_BASE 不会覆盖仍需保留的结果。

完成后至少检查合并特征 manifest、level/ 与 tog/ 稀疏矩阵、after_one.namefile.txt、标签文件、日志和各级 complete/failed 标志。当前功耗流程要求同一用例的 Level、TOG 与功耗标签行数一致；迁移到电压降风险任务后，还须检查风险矩阵的样本轴与特征行严格对齐。

3.2 代理筛选快速启动

cmd/run_timewindow_pipeline.csh 以数据集提取结果为输入，代理筛选流程由两个步骤组成：步骤 1 构造时间窗口样本；步骤 2 分别对各时间偏移和特征类型执行 MCP 或 XGBoost 代理筛选，并使用筛选后的代理训练独立 XGBoost 回归模型进行有效性验证，以判断代理集合是否保留了足够的功耗预测信息。

总控脚本启动示例：

```
cd /path/to/xgboost_timewindow

# 启用步骤2 代理有效性验证，并执行数据准备流程
setenv RUN_STEP2_VALIDATION 1
setenv STEP2_VALIDATION_COMBOS "tog_offset12,level_offset12"
csh cmd/run_timewindow_pipeline.csh gemmini
```

示例命令中 gemmini 是硬件标识参数，运行前应检查以下配置。

类别	变量或配置	检查内容
输入目录	REDUCTION_BASE、LEVEL_DIR、TOG_DIR、LABEL_DIR	确认特征和标签来自同一次数据集提取，用例名称、顺序及行数一致。
解释器与源码	PYTHON、TIMEWINDOW_SRC	确认使用已激活的 pro 环境，并能访问时间窗口和验证脚本。

特征名称	STAGE、NAMEFILE	默认使用 after_one.namefile.txt；名称数量和顺序须与输入矩阵列一致。
时间窗口	OFFSETS、DATA_TYPES、BOUNDARY_DISCARD、TEST_COUNT	核对偏移量、电平/翻转组合、边界丢弃长度和测试用例划分。
代理算法	PROXY_SELECTOR、MCP/XGBoost 参数	mcp 调用 MCP 筛选，xgb 或 xgboost 调用 XGBoost 筛选；只调整所选算法对应的参数。
并行设置	STEP1_PARALLEL_DSUB、STEP2_PARALLEL_DSUB、STEP1_JOBS、STEP2_JOBS	决定是否按用例/组合拆分任务，以及单任务内部并行度。
验证设置	RUN_STEP2_VALIDATION、STEP2_VALIDATION_COMBOS、STEP2_VALIDATION_CHUNK_SIZE	分别控制是否验证、参与验证的组合及拼接缓存每块行数。
调度与输出	OUTPUT_ROOT、DSUB_CMD、DSUB_QUEUE、DSUB_EXTRA_ARGS、POLL_SEC	确认结果目录、队列资源、提交命令和轮询周期。

代理筛选流程的两个步骤如下。

阶段	主要脚本	处理目的
步骤 1	step1_make_global_sample_index.py、step1_prepare_timewindow.py	划分训练/测试样本，并按偏移量和数据类型构造时间窗口特征。
步骤 2	step2_mcp_per_offset.py 或 step2_xgb_per_offset.py；step2_validation/xgb_power.py	对每个时间偏移与特征类型组合执行代理筛选，输出代理列及 manifest；随后重建代理特征并验证功耗预测误差和跳变跟踪能力。

运行脚本后，流程会自动调用 step1_make_global_sample_index.py 和 step1_prepare_timewindow.py 完成训练/测试样本划分及时间窗口特征构造，随后根据 PROXY_SELECTOR 调用 step2_mcp_per_offset.py 或 step2_xgb_per_offset.py，对各时间偏移与特征类型组合分别执行代理筛选；启用验证功能时，还会调用 step2_validation/xgb_power.py，读取时间窗口清单和代理清单，重建该组合对应的代理特征，并训练独立的 XGBoost 回归模型进行初步验证。

代理筛选结果保存在 step2_mcp/mcp_per_offset_manifest.json 或 step2_xgb_proxy/mcp_

per_offset_manifest.json 中，各组合的代理索引由其中的 selected_indices_file 给出。验证程序按 <hpotag>/<target>-bc<before_cycles>/ 动态生成结果目录，并在其 eval/ 子目录输出初步评价；其中 point_metrics.csv 给出平均绝对百分比误差（Mean Absolute Percentage Error，MAPE）、决定系数（Coefficient of Determination，R^2）和归一化均方根误差（Normalized Root Mean Squared Error，NRMSE）等预测指标。

迁移至电压降风险预测任务时，步骤 1 需将 LABEL_DIR 指向与特征对齐的 0/1 风险标签，并将连续功耗标签读取逻辑改为风险矩阵读取逻辑；现有 xgb_power.py 使用连续值功耗标签和回归目标，不能直接作为电压降风险验证程序，必须将标签接口、XGBoost 目标函数和评价指标替换为二分类或多输出风险预测实现。由于风险标签及空间映射尚未完成验证，上述内容仅说明流程的可迁移边界，不代表当前完整可执行的电压降代理筛选流程。

附录 A　数据集提取脚本

本附录按照 scripts/csv_reduction/run_fsdb_pipeline.csh 的实际处理顺序，说明数据集提取过程中调用的 C++ 可执行程序、Python 脚本及内联处理。

A.1　fsdb_to_tog_cache

功能描述： fsdb_to_tog_cache 用于读取 RTL 仿真生成的 FSDB 波形，将选定信号展开为位级列，并导出模型使用的初始电平数据与逐周期翻转信息。总控脚本通过 fsdb_list 逐项处理测试用例，每一项包含用例名称、FSDB 路径以及采样起止 tag。处理时先从首个测试用例导出可支持信号名，再按层次结构字符串筛选并生成 <HW>_signal_filter.txt。

数据转换阶段以测试用例为粒度顺序调用 fsdb_to_tog_cache。每个测试用例单独生成缓存目录和日志，并将成功用例的 tog_cache_manifest.json 加入后续合并列表。程序固定采用 sample-xor 方式计算相邻采样点的翻转信息，同时保存测量窗口起点的初始电平向量，后续可由“初始电平加逐周期翻转”恢复完整 Level 序列。

导出过程中，若某个位级线网在测量窗口的采样点上出现真实 FSDB X/Z，该列会从当前用例的输出矩阵中剔除，并记录到 manifest 的 removed_xz_names 字段；未落在采样点上的瞬态 X/Z 不触发剔除，NC 保持前值，NV 或无值按 0 处理。

示例命令：

1. 导出并筛选信号线网：

```
${TOOLS_ROOT}/fsdbapi/build/fsdb_to_tog_cache \
  --fsdb /data/coremark_1.fsdb \
  --list-signals work/all_signals.txt

# 按配置字符串筛选，并只保留第一列信号名
grep "gemmini.tile." work/all_signals.txt | \
  awk '{print $1}' > work/gemmini_signal_filter.txt
```

2. 数据提取与格式转换：

```
${TOOLS_ROOT}/fsdbapi/build/fsdb_to_tog_cache \
  --fsdb /data/coremark_1.fsdb \
  --output-dir work/cache_gemmini/coremark_1 \
  --dataset-name coremark_1 \
  --signal-list work/gemmini_signal_filter.txt \
  --step 2 --start 0 --end 50000 --end-inclusive \
  --raw-col-block 20000 \
  --tog-mode sample-xor --drop-first-tog-row \
  --max-cycles-per-file 1000
```

上述 start、end 和 step 均为 FSDB tag，实际物理时间由波形文件的 timescale 决定。

参数列表：

导出信号名参数：

参数	必需性	说明
--fsdb	必需	用于枚举信号的 RTL FSDB；总控流程使用 fsdb_list 中的首个测试用例。
--list-signals	必需	支持信号列表输出路径；设置后程序只枚举信号并退出。
SIGNAL_PREFIX	可选	总控脚本使用的信号匹配字符串；可配置一个或多个值，留空时跳过信号筛选。

数据提取转换参数：

参数	必需性	说明
--fsdb、--output-dir	必需	当前测试用例的 FSDB 路径与独立缓存输出目录。
--dataset-name	必需	由 fsdb_list 第一列确定的 cache_id。
--signal-list	条件使用	SIGNAL_PREFIX 非空时传入筛选后的信号名文件；未筛选时不设置。
--step	必需	由 STEP 设置采样间隔，单位为 FSDB tag，且必须大于 0。
--start、--end	必需	从当前 fsdb_list 条目读取测量窗口。
--end-inclusive	条件使用	END_INCL 为 1 时设置，用于包含 end 采样点。
--raw-col-block	必需	由 RAW_COL 指定程序内部一次处理的原始列规模。
--tog-mode	固定	当前总控脚本固定使用 sample-xor。
--drop-first-tog-row	条件使用	DROP_FIRST 为 1 时设置，用于删除首个翻转行。
--max-cycles-per-file	必需	限制单个 NPZ 文件保存的最大周期数；0 表示不主动按周期数分片。

输入输出件：

描述	类型	作用
RTL 仿真波形 *.fsdb	输入	提供信号层次、初始状态及测量窗口内的值变化。
fsdb_list	配置输入	定义用例名称、FSDB 路径和采样起止 tag。

all_signals.txt	可选中间输出	设置 SIGNAL_PREFIX 时，保存首个 FSDB 中枚举出的全部支持信号。
<HW>_signal_filter.txt	可选中间输入	保存匹配 SIGNAL_PREFIX 的信号名，并作为 --signal-list。
TOG 稀疏矩阵 *.npz	输出	保存所选线网的逐采样周期翻转特征，采用 CSC 稀疏格式并可按行分片。
初始 Level *.npy	输出	保存各保留列在测量窗口起点的初始电平，用于恢复 Level 序列。
tog_cache_manifest.json	输出	记录矩阵形状、分片路径、列名、X/Z 剔除结果、采样范围和缓存模式。
removed_xz_names	输出字段	记录在采样点上出现真实 X/Z 并被剔除的位级线网。
<cache_id>.log	输出	保存当前测试用例的标准输出和错误信息；转换失败时应首先检查该日志。

内存开销与限制分析：当前总控脚本为每个测试用例启动一次 fsdb_to_tog_cache，不创建独立的列块作业。RAW_COL 通过 --raw-col-block 控制程序内部一次处理的原始列规模，MAX_CYCLES_PER_FILE 通过 --max-cycles-per-file 控制输出 NPZ 的行分片规模。峰值内存主要由当前处理列的信号状态、采样结果、CSC 索引和值数组以及 NPZ 压缩缓冲构成，程序本身没有硬性内存预算参数。

建议先使用一个实际测试用例验证信号筛选结果和采样区间，再处理完整 fsdb_list。当设计列数较大时，应优先减小 RAW_COL；当测量窗口较长时，应适当减小 MAX_CYCLES_PER_FILE，避免单个 NPZ 文件过大。当前脚本按测试用例顺序执行，各用例完成后由后续 merge_tog_cache_manifests.py 沿样本方向合并缓存。

A.2　fsdb_to_power_label 与 TSV 转 NPY 过程

功能描述：功耗标签提取过程先调用 fsdb_to_power_label，按照统一的 STEP 导出功耗 TSV 和 JSON 元数据；随后直接执行内联 Python，读取 TSV 的 power 列，并保存为形状 $[N, 1]$、数据类型为 float32 的 NPY 标签。DROP_FIRST 为 1 且功耗记录数大于 1 时，内联程序删除首个采样值。

示例命令：

1. POWER_BIN 导出功耗 TSV：

```
${TOOLS_ROOT}/fsdbapi/build/fsdb_to_power_label \
  --fsdb /data/power/coremark_1.fsdb \
  --power-signal top.chip.power \
  --output-tsv output/gemmini_power/coremark_1.power.tsv \
  --output-meta output/gemmini_power/coremark_1.power.meta.json \
  --step 2 --start 0 --end 50000
```

当功耗信号名未知时，可在独立调试阶段枚举候选数值信号：

```
${TOOLS_ROOT}/fsdbapi/build/fsdb_to_power_label \
```

```
  --fsdb /data/power/coremark_1.fsdb \
  --list-signals work/power_numeric_signals.txt
```

2. 内联 TSV 转 NPY 程序：

```
import numpy as np, csv
with open('${POWER_DIR}/${cache_id}.power.tsv') as f:
    vals = [float(r['power']) for r in csv.DictReader(f, delimiter='\t')]
if ${DROP_FIRST} == 1 and len(vals) > 1:
    vals = vals[1:]
np.save('${POWER_DIR}/${cache_id}.npy', np.array(vals, dtype='float32').reshape(-1,1))
print(f'power:␣{len(vals)}␣rows␣→␣${cache_id}.npy')
```

该程序直接内联在 run_fsdb_pipeline.csh 的功耗用例循环中，无需保存为独立 Python 文件。独立调试时，应先将其中的 C Shell 变量替换为实际路径和值。

参数列表：

功耗用例与 POWER_BIN 参数：

参数	必需性	说明
power_list	可选	功耗用例列表；留空时跳过整个功耗标签提取步骤。
cache_id	列表项必需	用例标识，须与 fsdb_list 中的名称匹配。
power_fsdb	列表项必需	当前用例的 PTPX 功耗 FSDB 路径。
power_signal	列表项必需	目标数值功耗信号的完整层次名。
--output-tsv	必需	功耗序列 TSV 输出路径。
--output-meta	必需	采样设置和记录数的 JSON 元数据输出路径。
--step	必需	由全局 STEP 设置采样间隔，单位为 FSDB tag。
--start、--end	必需	按 cache_id 从 fsdb_list 中取得的采样起止 tag。
--list-signals	独立调试可选	枚举候选数值信号并退出；当前总控流程不调用。

内联 TSV 转 NPY 参数：

变量或字段	必需性	说明
POWER_DIR	必需	TSV、JSON 元数据和 NPY 标签所在目录。
cache_id	必需	用例标识，决定输入、输出文件名。
DROP_FIRST	必需	为 1 且记录数大于 1 时删除首个采样值。
TSV 字段 power	必需	转换为浮点标签的字段。

输出类型与形状	固定	float32，形状为 $[N, 1]$。

输入输出件：

描述	类型	作用
power_list	配置输入	定义用例标识、功耗 FSDB 路径和功耗信号名。
PTPX 功耗波形 *.fsdb	输入	提供目标功耗信号的时间序列。
fsdb_list 中的测量窗口与 STEP	配置输入	确定当前功耗用例的采样范围和采样间隔。
power_numeric_signals.txt	可选中间输出	独立枚举信号时保存候选数值信号；总控流程不生成。
<cache_id>.power.tsv	中间输出/输入	保存原始功耗序列，并作为内联 Python 的输入。
<cache_id>.power.meta.json	输出	记录功耗信号、采样范围、间隔和记录数。
<cache_id>.npy	输出	模型使用的二维 float32 功耗标签。
终端输出	运行信息	显示功耗提取失败、NPY 保存结果及最终标签行数。

内存开销与限制分析：功耗标签的列维度远小于特征矩阵，POWER_BIN 和内联 TSV 转 NPY 程序需要处理的数据规模也相对有限。相对于 512 GB 的设备内存，其内存开销很小，因此设备内存不会成为该脚本的主要限制。当前总控脚本按 power_list 中的用例顺序执行功耗提取，各用例输出到统一的 <HW>_power 目录，并通过 cache_id 区分文件。

A.3　merge_tog_cache_manifests.py

功能描述：特征转换完成后，若存在多个成功用例，则调用 merge_tog_cache_manifests.py，按照 fsdb_list 中成功用例的顺序沿样本行方向合并缓存，并在 MERGED_DIR/_work 下生成统一的 tog_cache_manifest.json。合并过程中需要检查各输入 manifest 的列块布局和列定义是否兼容，并对不同用例中剔除的 X/Z 列取并集。

示例命令：

```
python3 ${TOOLS_ROOT}/fsdbapi/merge_tog_cache_manifests.py \
  --output-dir output/gemmini_cache \
  --manifest work/cache_gemmini/coremark_1/_work/tog_cache_manifest.json \
  --manifest work/cache_gemmini/coremark_2/_work/tog_cache_manifest.json
```

参数列表：

参数	必需性	说明

--manifest	必需，可重复	成功用例的缓存 manifest；总控脚本按用例顺序重复添加该参数。
--output-dir	必需	合并缓存输出目录，即 MERGED_DIR。
--max-mem	当前未设置	程序采用默认合并内存预算。
--merge-mem-frac	当前未设置	程序采用默认的单次合并内存比例。
--max-cycles-per-file	当前未设置	程序根据默认策略决定是否进行行分片。

输入输出件：

描述	类型	作用
各用例 tog_cache_manifest.json	输入	定义各测试用例的 TOG 分片、初始 Level、列名、行数及 X/Z 清理信息。
各用例 TOG NPZ 与初始 Level NPY	输入	提供待纵向拼接的稀疏特征和 Level 恢复起点。
合并后的 TOG NPZ 分片	输出	按输入用例顺序保存全局样本矩阵。
块级检查点和元数据	中间输出	支持程序内部逐块合并及断点复用。
MERGED_DIR/_work/tog_cache_manifest.json	输出	供后续降维程序读取的统一缓存入口。
终端输出	运行信息	打印合并进度和最终 manifest 路径；失败时总控脚本立即退出。

内存开销与限制分析：脚本逐列块读取多个用例的稀疏矩阵，并沿样本行方向进行拼接，合并粒度与 A.1 中 fsdb_to_tog_cache 的原始信号分块粒度直接相关。峰值内存主要由当前列块的输入矩阵、拼接临时对象和压缩保存缓冲组成；列块包含的原始信号越多、合并后的周期数越大，所需内存通常越高。

根据现有数据的运行经验，合并约 5000 个原始信号、19 万周期的数据时，峰值内存约为 110 GB。其他数据规模可参照该结果，结合原始信号数量、展开后的位级列数、周期数和矩阵稀疏度估算内存需求。若设备内存无法容纳当前合并任务，应减小 A.1 中的 RAW_COL，以降低单个列块包含的原始信号数量和合并峰值内存；修改后需重新执行特征缓存导出，再进行多用例合并。当前总控脚本未显式传入 --max-mem、--merge-mem-frac 和 --max-cycles-per-file，因此使用程序默认值。

A.4 dimentionality_reduction_csv.py

功能描述：总控脚本使用合并后的 tog_cache_manifest.json 作为输入，调用 dimentionality_reduction_csv.py 完成无效信息剔除，并同时输出采用相同列选择的 Level 与 TOG 特征。当前流程以 TOG 数据统计列状态，清理 X/Z 无效列和全 0 列；对于全 1 列只保留一列代表列。相同的保留列索引随后应用到由“初始 Level 加 TOG”恢复的 Level 数据，以保证两类特征列严格对应。

脚本保存 X/Z 清理后、全 1 列处理后和最终阶段的数据。--disable-sim-reduce 明确关闭相似列处理，--force-row-shard-save 则要求阶段矩阵按行分片输出。

示例命令：

```
python3 -u ${TOOLS_ROOT}/dimentionality_reduction_csv.py \
  --input-cache-manifest output/gemmini_cache/_work/tog_cache_manifest.json \
  --output-dir output/gemmini_reduced \
  --output-meta output/gemmini_reduced/meta.json \
  --output-cols output/gemmini_reduced/final_cols.txt \
  --output-original-headers output/gemmini_reduced/original_headers.txt \
  --max-mem 180GB \
  --target-cols 500000 \
  --cache-format csc \
  --save-after-xz-clean \
  --save-after-one \
  --save-final \
  --disable-sim-reduce \
  --duplicate-mode hash-only \
  --input-data-type level \
  --force-row-shard-save \
  --snapshot-row-block-rows 1000
```

参数列表：

参数	必需性	说明
--input-cache-manifest	必需	合并后的 MERGED_DIR/_work/tog_cache_manifest.json。
--output-dir	必需	降维结果根目录，即 REDUCE_DIR。
--output-meta	必需	全流程统计信息和处理结果 JSON。
--output-cols	必需	最终保留列名的配置输出路径。
--output-original-headers	必需	原始展开列名输出路径。
--max-mem	必需	由 MAX_MEM 设置内存预算，当前示例为 180 GB。
--target-cols	必需	由 TARGET_COLS 设置目标列数提示。
--cache-format	固定	当前流程使用 CSC 格式读取和统计列。
--save-after-xz-clean	固定设置	保存 X/Z 清理后的阶段矩阵。
--save-after-one	固定设置	保存全 0 列剔除及全 1 列合并后的阶段矩阵。
--save-final	固定设置	保存最终阶段矩阵。
--disable-sim-reduce	固定设置	关闭相似列降维。
--duplicate-modehash-only	固定设置	设置程序内部对应处理模式，具体算法不在本文展开。
--input-data-typelevel	固定设置	同时生成 Level 与 TOG 输出，并对两类数据应用相同列选择。
--force-row-shard-save	固定设置	强制将阶段结果按样本行分片保存。

--snapshot-row-block-rows	必需	由 MAX_CYCLES_PER_FILE 设置每个快照分片的最大周期数。

输入输出件：

描述	类型	作用
合并后的 tog_cache_manifest.json	输入	提供 TOG 分片、初始 Level 文件、列名和样本映射。
TOG NPZ 与初始 Level NPY	输入	用于统计有效列并恢复完整 Level 序列。
level/ 目录	输出	保存各阶段的 Level 稀疏矩阵、分片 manifest 和列名。
tog/ 目录	输出	保存各阶段的 TOG 稀疏矩阵、分片 manifest 和列名。
after_xz_clean 阶段	输出	保存 X/Z 无效列清理后的特征。
after_one 阶段	输出	保存全 0 列剔除并仅保留一列全 1 列后的特征。
final 阶段	输出	保存当前配置下的最终特征矩阵。
meta.json	输出	记录各阶段列数、清理数量、输入类型和处理耗时。
最终列名与原始表头文件	输出	记录模型输入列及其与原始信号的对应关系。
reduction.log	输出	由总控脚本通过 tee 保存降维进度和异常信息。

内存开销与限制分析：降维阶段的主要内存开销来自稀疏矩阵读取、逐列统计、Level 恢复、CSC/CSR 格式转换以及阶段快照保存。当前总控脚本将 MAX_MEM 设置为 180 GB，并通过 --force-row-shard-save 和 --snapshot-row-block-rows 控制快照规模。在 512 GB 内存设备上，该配置为运行时临时对象和系统开销保留了额外空间，但 --max-mem 属于程序预算而非操作系统级硬限制，稀疏格式转换和压缩保存仍可能产生短时额外开销。建议根据实际矩阵列数和稀疏度观察 reduction.log，必要时减小快照行数。

附录 B　代理筛选脚本

本附录按照 cmd/run_timewindow_pipeline.csh 中实际调用顺序，介绍代理筛选流程的关键脚本。

B.1　step1_make_global_sample_index.py

功能描述：step1_make_global_sample_index.py 读取 Level、TOG 和功耗标签目录，取得三者共有的测试用例，划分训练用例与测试用例，并在统一有效边界内生成逐用例采样索引。全部偏移和数据类型组合复用同一份索引，从而保证不同组合具有一致的标签样本集合和样本顺序。

示例命令：

```
${PYTHON} src/timewindow/step1_make_global_sample_index.py \
  --level-dir reduced/level \
  --tog-dir reduced/tog \
  --label-dir power \
  --output-dir run/step1_shifted/sample_index \
  --offsets 12,11,10,9,8,7,6 \
  --boundary-discard 12 \
  --test-count 1 \
  --stage after_one \
  --sample-rows-total 0 \
  --sample-rows-per-file 0 \
  --sample-bins 20 \
  --sample-min-per-bin 1 \
  --sample-min-per-case 1 \
  --sample-test-ratio -1 \
  --sample-seed 108
```

参数列表：

参数	必需性	说明
--level-dir、--tog-dir	必需	Level 与 TOG 阶段矩阵目录。
--label-dir、--output-dir	必需	功耗标签目录和采样索引输出目录。
--offsets	可选	逗号分隔的偏移列表，默认 12 至 6。
--boundary-discard	可选	首尾边界丢弃行数；未设置时取最大偏移。
--test-count	可选	从共有用例末尾划入测试集的用例数量，默认 1。

--stage	可选	读取的特征阶段名，默认 after_one。
--sample-rows-total	可选	训练集全局采样上限；0 表示不设置上限。
--sample-rows-per-file	可选	单个用例采样上限；0 表示保留全部有效行。
--sample-bins	可选	功耗标签分位数分箱数量，默认 20。
--sample-min-per-bin、--sample-min-per-case	可选	每个非空功耗区间及每个用例的最低采样保障。
--sample-test-ratio	可选	测试集采样比例；负数表示沿用训练集采样规则。
--sample-seed	可选	采样随机种子，默认 108。

输入输出件：

描述	类型	作用
Level、TOG 阶段矩阵或分片 manifest	输入	确定各用例可用的特征行数。
<stem>.npy 功耗标签	输入	确定有效标签范围并执行功耗分层采样。
sample_index_manifest.json	输出	记录训练/测试划分、采样设置及逐用例索引路径。
train_stems.txt、test_stems.txt	输出	保存互斥的训练与测试用例列表。
逐用例索引 NPY	输出	保存后续时间窗口构造使用的统一标签行索引。
complete/failed/status	调度输出	标识索引作业是否成功完成。

内存开销与限制分析：程序需要读取当前用例的标签并保存候选位置、分箱结果和最终索引，内存主要随标签行数线性增长，通常远小于特征矩阵处理开销，因此该脚本无需内存限制条件。

B.2 step1_prepare_timewindow.py

功能描述：step1_prepare_timewindow.py 根据采样索引，按测试用例、偏移和数据类型构造时间窗口训练数据。对于标签行 i 和偏移 k，TOG 特征取行 $i-k$，Level 特征取行 $i+1-k$。在默认并行模式下，总控脚本为每个“用例与组合”生成一个独立任务，调用程序时通过 --include-stems、单一 --offsets 和单一 --data-types 限定处理范围；所有局部 manifest 完成后，再由总控脚本统一合并。

示例命令：

```
${PYTHON} src/timewindow/step1_prepare_timewindow.py \
  --level-dir reduced/level \
  --tog-dir reduced/tog \
```

```
--label-dir power \
--output-dir run/step1_shifted/_parts/coremark_1/tog_offset12 \
--offsets 12 \
--data-types tog \
--boundary-discard 12 \
--test-count 1 \
--stage after_one \
--sample-rows-per-file 0 \
--sample-bins 20 \
--sample-min-per-bin 1 \
--sample-seed 108 \
--jobs 1 \
--include-stems coremark_1 \
--sample-index-manifest run/step1_shifted/sample_index/sample_index_manifest.json \
--compress-output 0 \
--skip-existing 1
```

参数列表：

参数	必需性	说明
--level-dir、--tog-dir、--label-dir	必需	输入 Level、TOG 和功耗标签目录。
--output-dir	必需	当前任务的组合数据输出目录。
--offsets、--data-types	可选	偏移列表及 tog,level 数据类型列表。
--boundary-discard、--test-count、--stage	可选	边界丢弃量、测试用例数和输入阶段名。
--sample-index-manifest	建议设置	全局采样索引 manifest；总控并行流程固定传入。
--include-stems	可选	仅处理指定的逗号分隔用例；总控并行流程每次传入一个用例。
--sample-rows-per-file、--sample-bins、--sample-min-per-bin、--sample-seed	可选	未使用全局索引时的本地采样参数。
--jobs	可选	程序内部组合并行数；拆分调度任务中固定为 1。
--compress-output	可选	是否压缩 NPZ；0 降低保存时间和部分临时开销，1 减少磁盘占用。
--skip-existing	可选	是否复用已存在的特征和标签文件，使用 0/1 表示。

输入输出件：

描述	类型	作用
Level 或 TOG 阶段矩阵	输入	提供当前用例与数据类型的原始特征。
功耗标签 NPY	输入	提供当前用例的监督标签。
sample_index_manifest.json 及索引 NPY	输入	统一各组合使用的标签样本位置和顺序。
<stem>.feature.npz	输出	保存当前组合的时间对齐稀疏特征。
<stem>.label.npy	输出	保存与特征逐行对应的功耗标签。
训练/测试 filelist	输出	记录当前组合的训练与测试文件。
局部及全局 timewindow_manifest.json	输出	记录组合、偏移、数据类型、矩阵形状和文件路径。

内存开销与限制分析：单任务峰值至少包括当前用例的源稀疏矩阵、采样索引切片、输出组合矩阵和保存缓冲。--jobs 大于 1 时，多个组合会并发加载数据，峰值内存近似按工作进程数增加；默认拆分调度将每个任务的 --jobs 固定为 1。关闭输出压缩可减少压缩缓冲和运行时间，但会增加磁盘占用。脚本没有硬性内存上限，超大用例应通过调度资源限制并控制同时运行的任务数。

B.3 step2_mcp_per_offset.py

功能描述：step2_mcp_per_offset.py 是步骤 2 的默认代理筛选程序。脚本从步骤 1 manifest 读取指定组合的训练和测试特征，使用 MCP 筛选代理列，并通过 Ridge 回归对所选代理进行初步评价；当 MCP 求解不可用或未得到有效代理时，程序使用内部回退策略生成候选列。默认并行模式下，总控脚本为每个组合提交一个任务，完成后将局部 manifest 合并为 step2_mcp/mcp_per_offset_manifest.json。

示例命令：

```
${PYTHON} src/timewindow/step2_mcp_per_offset.py \
  --manifest run/step1_shifted/timewindow_manifest.json \
  --namefile reduced/level/after_one.namefile.txt \
  --alpha 50 \
  --gamma 3 \
  --alpha-l2 8 \
  --batch 4 \
  --jobs 1 \
  --include-combos tog_offset12 \
  --output-dir run/step2_mcp/_parts/tog_offset12
```

参数列表：

参数	必需性	说明
--manifest	必需	步骤 1 生成的 timewindow_manifest.json。

--output-dir	必需	当前组合或完整步骤 2 结果的输出目录。
--namefile	可选	原始特征名文件，用于输出可追溯的代理名称。
--alpha、--gamma	可选	MCP 正则强度和凹度参数，默认 50 和 3。
--alpha-l2	可选	Ridge 初评使用的 L2 正则强度，默认 8。
--batch	可选	每批拼接的训练文件数量，默认 4。
--top-k	可选	回退策略的代理数量，程序默认 200；当前总控命令未显式设置。
--jobs	可选	组合内部并行数；拆分调度任务中固定为 1。
--include-combos	可选	仅处理指定的逗号分隔组合。

输入输出件：

描述	类型	作用
timewindow_manifest.json	输入	提供组合名称及训练/测试特征和标签路径。
组合 feature.npz 与 label.npy	输入	提供代理筛选和初步评价数据。
NAMEFILE	可选输入	将代理列索引映射为原始信号名称。
selected_indices.txt	输出	保存当前组合选中的代理列索引。
代理名称、模型和评价文件	输出	记录代理的信号映射以及 Ridge 初步评价结果。
mcp_per_offset_manifest.json	输出	记录各组合的代理索引文件、评价文件和完成状态。

内存开销与限制分析：MCP 代理筛选的内存开销与输入矩阵规模、训练批次和组合并发数有关。具体的内存模型、限制条件及参数配置建议将在“代理筛选算法说明”文档中详细分析，本文不再展开。

B.4 step2_xgb_per_offset.py

功能描述：step2_xgb_per_offset.py 是步骤 2 的可替换实现。脚本针对每个组合训练 XGBoost 模型，根据特征重要性选择前 K 个代理，并使用独立测试文件生成初步评价。其输出 manifest 沿用 mcp_per_offset_manifest.json 名称，以便验证入口采用统一接口。将总控变量 PROXY_SELECTOR 设置为 xgb 或 xgboost 即可启用该程序。

示例命令：

```
${PYTHON} src/timewindow/step2_xgb_per_offset.py \
  --manifest run/step1_shifted/timewindow_manifest.json \
```

```
--namefile reduced/level/after_one.namefile.txt \
--batch 4 \
--top-k 200 \
--rounds 50 \
--max-depth 4 \
--eta 0.08 \
--subsample 0.9 \
--colsample-bytree 0.8 \
--nthread 1 \
--jobs 1 \
--include-combos tog_offset12 \
--output-dir run/step2_xgb_proxy/_parts/tog_offset12
```

参数列表：

参数	必需性	说明
--manifest、--output-dir	必需	步骤 1 manifest 和步骤 2 输出目录。
--namefile、--include-combos	可选	特征名文件和组合过滤列表。
--batch	可选	每批训练文件数量。
--top-k	可选	每个组合保留的代理数量，当前总控默认 200。
--rounds	可选	XGBoost 提升轮数，当前总控默认 50。
--eta、--max-depth	可选	学习率和树深。
--subsample、--colsample-bytree	可选	每轮训练的样本和特征采样比例。
--nthread	可选	单个 XGBoost 模型使用的线程数。
--jobs	可选	组合内部并行数；拆分调度任务中固定为 1。

输入输出件：

描述	类型	作用
timewindow_manifest.json	输入	提供组合名称及训练/测试文件路径。
组合 feature.npz 与 label.npy	输入	用于训练代理筛选模型和计算测试指标。
selected_indices.txt	输出	保存按特征重要性选出的代理列索引。
代理名称、XGBoost 模型和评价文件	输出	保存代理映射、模型及独立测试初评。
mcp_per_offset_manifest.json	输出	使用统一格式记录各组合的代理索引和评价结果。

内存开销与限制分析：XGBoost 代理筛选的内存开销与输入矩阵规模、训练批次、模型工作区、线程数和组合并发数有关。具体的内存模型、限制条件及参数配置建议将在“代理筛选算法说明”文档中详细分析，本文不再展开。

B.5 xgb_power.py

功能描述：xgb_power.py 用于验证步骤 2 输出的代理特征。脚本读取步骤 1 timewindow_manifest.json 和步骤 2 mcp_per_offset_manifest.json，根据 --combos 按给定顺序选择组合；每个组合使用步骤 1 manifest 中对应的特征文件和标签文件，并依据步骤 2 manifest 中的 selected_indices_file 截取代理列。多个组合的代理特征按列拼接，标签和样本顺序必须保持一致。

运行后，脚本自动调用数据集构造模块，将各组合代理特征写入分块缓存；若未设置 --build-dataset-only，则继续训练独立 XGBoost 回归模型，并调用评价模块生成逐点误差和功耗跳变误差。代理索引及其信号映射保存在动态生成的 indices/ 目录，当前组合代理的初步评价保存在同一实验目录的 eval/evaluation_summary.json。实验目录采用 <hpotag>/<target>-bc<before_cycles>/ 形式，由参数共同决定，并非固定目录。

示例命令：

```
cd run/step2_validation_legacy_xgb
${PYTHON} src/timewindow/step2_validation/xgb_power.py \
  --tw-manifest run/step1_shifted/timewindow_manifest.json \
  --proxy-manifest run/step2_mcp/mcp_per_offset_manifest.json \
  --namefile reduced/level/after_one.namefile.txt \
  --combos "tog_offset12,level_offset12" \
  --cache-dir run/step2_validation_legacy_xgb/dataset_cache \
  --build-block-size 200 \
  --hpotag legacy_xgb_validation
```

若只检查代理缓存、组合顺序和 manifest 一致性，可添加 --build-dataset-only；此时程序在数据集缓存构造完成后停止，不训练模型，也不生成预测评价文件。

参数列表：

参数	必需性	说明
--tw-manifest、--proxy-manifest	必需	步骤 1 与步骤 2 的统一 manifest。
--namefile	可选	原始信号名文件，用于输出代理来源映射。
--combos	可选	逗号分隔的组合名称及拼接顺序；留空时使用步骤 1 中的全部可用组合。
--cache-dir	可选	代理特征数据集缓存目录。
--build-block-size	可选	每个缓存分片的样本行数；较小值降低组合拼接暂存内存但增加磁盘 I/O。
--data-dtype	可选	缓存特征类型，可取 float32 或 float64。

--build-dataset-only	可选	只构建代理数据缓存，不训练和评价。
--rounds	可选	XGBoost 提升轮数，默认 200。
--length、--before-cycles	可选	序列长度和预测前周期数；后者也参与动态输出目录命名。
--label-start-cycle、--label-scale	可选	标签起始周期和数值缩放。
--target、--hpotag	可选	预测目标名称和实验标签，共同决定输出路径。
--fold、--shuffle-seed、--fixed-test-last-file	可选	训练/验证/测试划分设置。
--tree-method、--max-bin、--max-depth、--max-leaves	可选	XGBoost 树构造参数。
--learning-rate、--subsample、--colsample-bytree、--colsample-bynode	可选	学习率及行列采样参数。
--nthread、--seed、--device	可选	线程、随机种子和可选计算设备。

输入输出件：

描述	类型	作用
timewindow_manifest.json	输入	提供各组合的步骤 1 特征、标签和样本顺序。
mcp_per_offset_manifest.json	输入	提供各组合的 selected_indices_file 和代理筛选元数据。
组合特征、标签及代理索引	输入	构造多组合代理数据集。
part_*.npz、标签与样本元数据 NPY	缓存输出	保存分块代理特征及其监督标签、用例和周期信息。
dataset.meta.json	缓存输出	记录组合顺序、代理来源、列范围和缓存分片。
动态 indices/ 目录	输出	保存代理索引、原始列和信号名称映射。
动态 result/、pred/ 目录	训练输出	保存模型结果、预测 CSV 和重要性信息。

动态 eval/evaluation_summary.json	评价输出	汇总平均绝对百分比误差（Mean Absolute Percentage Error，MAPE）、决定系数（Coefficient of Determination，R^2）、归一化均方根误差（Normalized Root Mean Squared Error，NRMSE）及功耗跳变指标。

内存开销与限制分析：最终特征宽度等于各组合代理数量之和，组合数和每组合代理数会近似线性增加缓存与训练数据规模。数据集构造至少需要加载当前组合的源矩阵并截取代理列，训练阶段还需容纳训练、验证数据和 XGBoost 直方图工作区。建议优先使用 float32，控制组合数量、代理数量和线程数。在硬件内存受限时可以考虑减少时间窗长度、降低初筛代理数量等方法降低数据维度，以此满足内存限制要求。

附录 C　相关文档

文档名	主要内容
《代理筛选算法说明》	说明时间窗口样本构造、MCP/XGBoost 代理筛选、代理组合及验证算法。
《功耗与电压降风险模型数据集采集说明》	说明功耗预测与电压降风险预测任务所需原始数据、采样口径、命名组织和交付要求。